Zero Trust Network Access (ZTNA) with

Zscaler: Securing User-to-Application Connectivity

Project Details

Project Title: Zero Trust Network Access (ZTNA) with Zscaler: Securing User-to-Application Connectivity

Domain: Cybersecurity / Web Application Security

Objective:

The objective of this project is to identify, analyse, and understand security vulnerabilities in web applications by conducting vulnerability assessments with security testing tools. The project aims to simulate real-world security testing scenarios in which vulnerabilities such as injection attacks, misconfigurations, and information disclosure may exist in web applications.

Practice Website Used:

http://www.itsecgames.com

The selected website hosts bWAPP (Buggy Web Application), an intentionally vulnerable web application used for cybersecurity training and penetration testing. It contains multiple security flaws that enable students and researchers to understand how vulnerabilities arise and how attackers exploit them.

Security Tools Used:

- **Nessus** – Vulnerability scanning tool used to identify system and network vulnerabilities
- **Nmap** – Network scanning tool used to detect open ports and running services
- **SecurityHeaders.com** – Tool used to analyse HTTP security headers
- **SSL Labs** – A tool used to analyse SSL certificate configuration
- **Browser Developer Tools** – Used to inspect HTTP response headers and network responses

Testing Methodology:

The vulnerability assessment process followed these steps:

1. **Target Identification**
   The target web application was identified and analysed to understand its architecture and functionality.

2. **Information Gathering**
   Basic information about the web server, technologies used, and network services was collected.

3. **Vulnerability Scanning**
   Automated security scanning tools were used to detect potential vulnerabilities in the application.

4. **Manual Testing**
   Manual testing techniques were used to confirm vulnerabilities, including injection flaws and configuration issues.

5. **Analysis and Documentation**
   Each vulnerability was analysed and documented, including a description, impact, reproduction steps, and mitigation recommendations.

Abstract:

Web applications have become an essential component of modern digital infrastructure, supporting services such as online education, banking systems, e-commerce platforms, and enterprise applications. As the number of web applications continues to grow, so does the number of cyber threats targeting these systems. Attackers frequently exploit vulnerabilities in web applications to gain unauthorised access, steal sensitive information, or disrupt services.

Vulnerability assessment plays a critical role in identifying security weaknesses before malicious actors can exploit them. By systematically analysing web applications, organisations can detect security flaws and implement appropriate mitigation strategies to strengthen their security posture.

This project focuses on performing a vulnerability assessment on a web application using security testing tools and manual analysis techniques. The practice website http://www.itsecgames.com, which hosts the bWAPP (Buggy Web Application) platform, was selected as the testing environment. This platform contains multiple intentionally vulnerable modules designed to demonstrate common web security flaws.

During testing, several vulnerabilities were identified, including missing HTTP security headers, SSL/TLS configuration issues, SQL injection and command injection, open network ports, and information disclosure. Each vulnerability was analysed and classified using Common Weakness Enumeration (CWE) identifiers and OWASP Top 10 security categories, which are widely recognised standards for web application security.

The findings from this project highlight the importance of secure coding practices, proper server configuration, and regular vulnerability assessments in protecting web applications from cyber threats. By understanding how these vulnerabilities occur and how they can be mitigated, developers and security professionals can build more secure and resilient applications.

Use Case Scenarios:

The following scenarios illustrate how vulnerability assessment techniques can be applied in real-world cybersecurity environments.

Scenario 1: Web Application Security Assessment

A cybersecurity analyst is assigned the task of evaluating the security posture of a web application before it is deployed to production. The analyst conducts vulnerability scanning with tools such as Nessus and Nmap to identify potential security vulnerabilities.

During the scanning process, the analyst identifies several vulnerabilities, including missing security headers, outdated SSL configurations, and open network ports. These findings help the development team implement security fixes before the application is released to users.

Output:

- Security vulnerabilities identified in the web application
- Risk levels analysed and documented
- Security recommendations provided to developers

Scenario 2: Identifying Injection Vulnerabilities

A security tester analyses the input fields of a web application to determine whether the application properly validates user input. By entering specially crafted inputs into the search and login fields, the tester identifies SQL injection vulnerabilities.

These vulnerabilities demonstrate how attackers may manipulate database queries to retrieve unauthorised data or bypass authentication mechanisms.

Output:

- Injection vulnerabilities detected
- Sensitive database information exposure risk identified
- Secure coding recommendations provided

Scenario 3: Detecting Remote Command Execution

The security analyst tests application parameters that interact with system commands. During testing, the analyst discovers that user inputs are passed directly into system commands without proper validation.

This vulnerability allows attackers to execute system commands remotely, potentially giving them control over the server.

Output:

- Remote command execution vulnerability identified
- Risk of server compromise analysed
- Input validation and filtering recommendations provided

Scenario 4: Network Service Exposure Analysis

The analyst performs network scanning using tools such as Nmap to identify open ports and running services on the target server. The scan results reveal that several services are accessible from external networks.

These exposed services may provide potential entry points for attackers if they are not properly secured.

Output:

- Open network ports detected
- Running services identified
- Recommendations provided for firewall configuration and service hardening

Scenario 5: Information Disclosure Detection

During the analysis of HTTP response headers, the security analyst discovers that the web server discloses details of its software configuration, including the server type and version.

This information can assist attackers in identifying known vulnerabilities associated with the server software.

Output:

- Server technology information identified
- Potential attack surface analysed
- Recommendations provided to limit information disclosure

STAGE: 1

Title of the Project: Zero Trust Network Access (ZTNA) with Zscaler: Securing User-to-Application Connectivity

Overview:

In today's digital world, web applications play a significant role in providing online services for organisations, businesses, educational institutions, and government systems. These applications allow users to access services such as online banking, e-commerce platforms, cloud services, academic portals, and enterprise management systems. However, because web applications are accessible via the internet, they are also common targets for cyber attackers who attempt to exploit vulnerabilities to gain unauthorised access, steal confidential data, manipulate system functionality, or disrupt services.

Web application vulnerabilities often arise due to improper input validation, insecure configurations, outdated software components, weak authentication mechanisms, and insufficient security controls within the application architecture. Attackers can exploit these weaknesses using techniques such as SQL injection, command injection, cross-site scripting, and privilege escalation. If these vulnerabilities remain undetected, they may lead to serious consequences, including data breaches, financial losses, reputation damage, and legal implications for organisations.

To mitigate these risks, vulnerability assessment is an essential cybersecurity process that focuses on identifying, analysing, and evaluating security weaknesses within systems and applications. Vulnerability assessment helps organisations proactively detect potential security flaws before attackers exploit them. By performing systematic security testing, organisations can strengthen their defences and improve the overall security posture of their systems.

The primary objective of this stage of the project is to perform a vulnerability assessment on a web application and identify common security weaknesses using both automated security tools and manual testing techniques. The testing was conducted on the practice website http://www.itsecgames.com, which hosts the bWAPP (Buggy Web Application) platform. The bWAPP platform is intentionally designed with numerous security vulnerabilities. It is widely used in education to help students and cybersecurity professionals understand how different types of web application attacks occur in real-world environments.

During testing, several security analysis techniques were employed to evaluate the web application's security posture. These techniques include analysing HTTP response headers, checking SSL/TLS certificate configurations, performing network port scanning, identifying injection vulnerabilities, and examining server configuration details. Various security tools, including the Nessus vulnerability scanner, Nmap network scanner, SSL Labs, Security Headers Analyser, and browser developer tools, were used to detect and analyse vulnerabilities in the target application.

The vulnerabilities identified during this assessment include missing HTTP security headers, invalid SSL certificate configuration, SQL injection vulnerabilities, command injection vulnerabilities, open ports and exposed services, server information disclosure, and technology fingerprinting vulnerabilities. Each vulnerability was analysed in detail and categorised using the Common Weakness Enumeration (CWE) identifiers and the OWASP Top 10 security risks, which represent the most critical web application security threats recognised by the global cybersecurity community.

Understanding these vulnerabilities is essential for developers, system administrators, and security professionals because it highlights how improper configurations and insecure coding practices can expose applications to cyber attacks. By identifying these weaknesses and implementing appropriate mitigation strategies, organisations can significantly reduce the risk of exploitation and ensure that their web applications remain secure, reliable, and resilient against evolving cyber threats.

Overall, this stage of the project provides a foundational understanding of web application security testing and demonstrates how vulnerability assessment techniques can be used to detect security flaws in web applications. The knowledge gained from this analysis will support further stages of the project, including advanced vulnerability scanning, risk assessment, and implementation of secure access mechanisms.

Figure 1: Overview of Web Application Vulnerability Assessment Process

List of Teammates

S.No.	Name	College Name	Contact
1	Sri Vaishnavi S	KPR College Of Arts Science & Research	9597974341
2	Akilan K	KPR College Of Arts Science & Research	9444611108
3	Dharaneesh SS	KPR College Of Arts Science & Research	9944396150
4	Ragul R	KPR College Of Arts Science & Research	9677944958

List of Vulnerabilities

Practice Website:
http://www.itsecgames.com

The following table lists the vulnerabilities identified during the security assessment of the target web application. These vulnerabilities were discovered using a combination of automated vulnerability scanning tools and manual security testing techniques. Each vulnerability is classified using Common Weakness Enumeration (CWE) identifiers and mapped to the OWASP Top 10 security categories, which represent the most critical web application security risks.

S.No	Vulnerability Name	CWE ID	OWASP Category
1	Missing Security Headers	CWE-693	A05:2021 – Security Misconfiguration
2	Invalid SSL Certificate	CWE-295	A07:2021 – Identification and Authentication Failures
3	SQL Injection (GET Search)	CWE-89	A03:2021 – Injection
4	Command Injection (Remote Command Execution)	CWE-78	A03:2021 – Injection
5	Open Ports and Service Exposure	CWE-284	A05:2021 – Security Misconfiguration
6	Server Information Disclosure	CWE-200	A05:2021 – Security Misconfiguration
7	Technology Information Disclosure	CWE-200	A05:2021 – Security Misconfiguration
8	Cross-Site Scripting (XSS)	CWE-79	A03:2021 – Injection
9	Insecure Cookie Configuration	CWE-614	A05:2021 – Security Misconfiguration
10	Clickjacking Vulnerability	CWE-1021	A05:2021 – Security Misconfiguration

Vulnerability Analysis

This section provides a detailed analysis of the vulnerabilities identified during the security testing of the target web application. Each vulnerability is explained with its technical description, potential impact on the system, and recommended mitigation strategies to prevent exploitation.

The analysis includes vulnerabilities related to web application configuration, input validation, network exposure, and server information disclosure. Understanding these vulnerabilities helps developers and security professionals implement proper security controls to protect applications from cyber attacks.

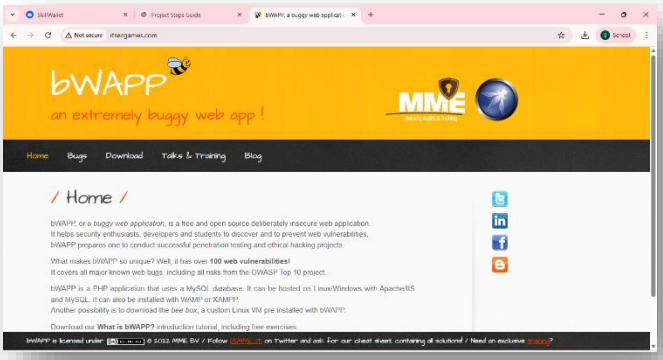

Figure 2: bWAPP (Buggy Web Application) Practice Website Homepage

The vulnerabilities identified in this assessment include:

- Missing Security Headers
- Invalid SSL Certificate Configuration
- SQL Injection Vulnerability
- Command Injection Vulnerability
- Open Network Ports and Service Exposure
- Server Information Disclosure
- Technology Information Disclosure
- Cross-Site Scripting (XSS)
- Insecure Cookie Configuration
- Clickjacking Vulnerability

Each of these vulnerabilities is explained in the following subsections.

1. Vulnerability Name: Missing Security Headers

CWE: CWE-693 – Protection Mechanism Failure

OWASP Category: A05:2021 – Security Misconfiguration

Description:

- Security headers are HTTP response headers used to enhance the security of web applications by instructing web browsers how to behave when handling web content. These headers help prevent various types of client-side attacks such as cross-site scripting (XSS), clickjacking, and MIME-type sniffing.
- During the vulnerability assessment, the website was analysed using a security header analysis tool. The results showed that several essential security headers were missing from the server response, including X-Frame-Options, X-Content-Type-Options, and Content-Security-Policy.
- Without these headers, browsers cannot enforce strict security rules when rendering web content. This increases the risk of malicious scripts being executed within the user's browser environment.

Business Impact:

Missing security headers may allow attackers to manipulate how web content is displayed in the browser. Attackers can perform clickjacking attacks or inject malicious scripts into web pages, potentially leading to data theft or unauthorised user actions.

Evidence:

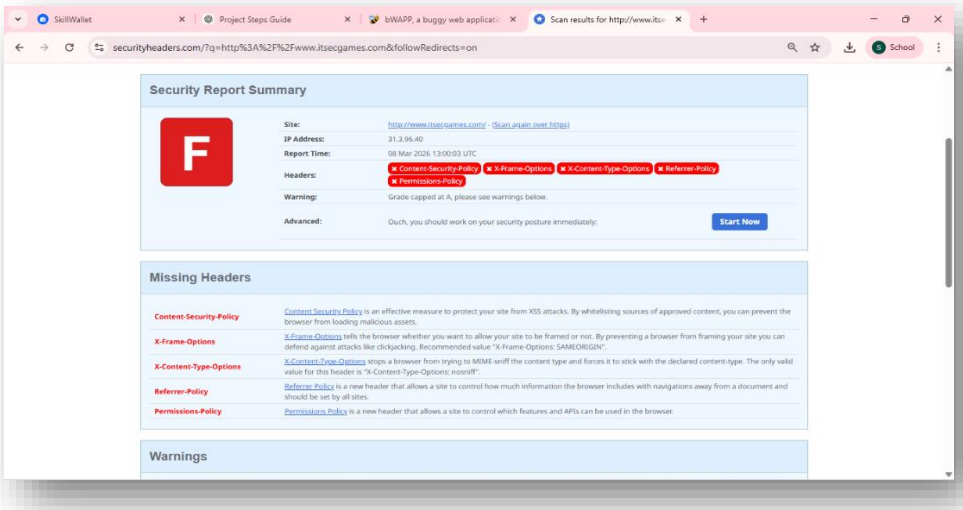

Figure 3: Security header analysis showing missing HTTP security headers on the target web application.

Recommendation:

- X-Frame-Options
- X-Content-Type-Options
- Content-Security-Policy
- Referrer-Policy

2. Vulnerability Name: Invalid SSL Certificate

CWE: CWE-295 – Improper Certificate Validation

OWASP Category: A07:2021 – Identification and Authentication Failures

Description:

- SSL certificates are used to establish secure, encrypted communication between a web server and a client. They ensure that data transmitted between users and the server cannot be intercepted or modified by unauthorised parties.
- During the SSL configuration analysis, it was observed that the website's SSL certificate configuration contains validation issues. An invalid or improperly configured certificate may cause browsers to display warnings indicating that the connection may not be secure.
- Improper certificate validation can allow attackers to impersonate legitimate servers and intercept communication between users and the application.

Business Impact:

If attackers exploit this vulnerability, they may perform **Man-in-the-Middle (MITM) attacks**, allowing them to intercept sensitive information such as login credentials, personal data, or financial information.

Evidence:

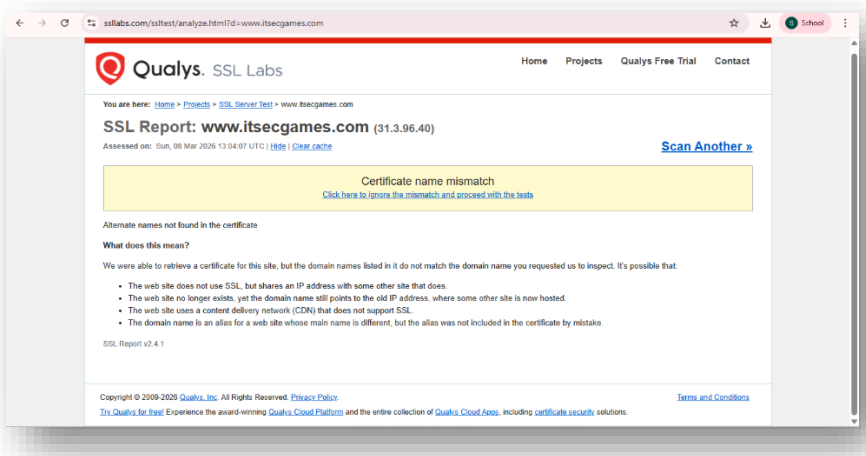

Figure 4: SSL configuration analysis displaying invalid or mismatched SSL certificate details.

Recommendation:

- Use SSL certificates issued by trusted Certificate Authorities
- Ensure the certificate domain matches the server domain
- Regularly update SSL certificate configurations
- Implement secure TLS versions (TLS 1.2 or TLS 1.3)

3. Vulnerability Name: SQL Injection (GET Search)

CWE: CWE-89 – SQL Injection

OWASP Category: A03:2021 – Injection

Description:

- SQL Injection is a vulnerability that occurs when user input is not properly validated before being used in database queries. Attackers can insert malicious SQL statements into input fields, allowing them to manipulate database queries.
- During testing of the search functionality, it was observed that the application does not properly sanitise user inputs. By inserting specially crafted SQL queries into the search parameter, attackers may manipulate database queries and retrieve unauthorised information from the database.

Business Impact:

SQL injection vulnerabilities can allow attackers to access sensitive database information, modify records, bypass authentication mechanisms, or even gain administrative privileges.

Evidence:

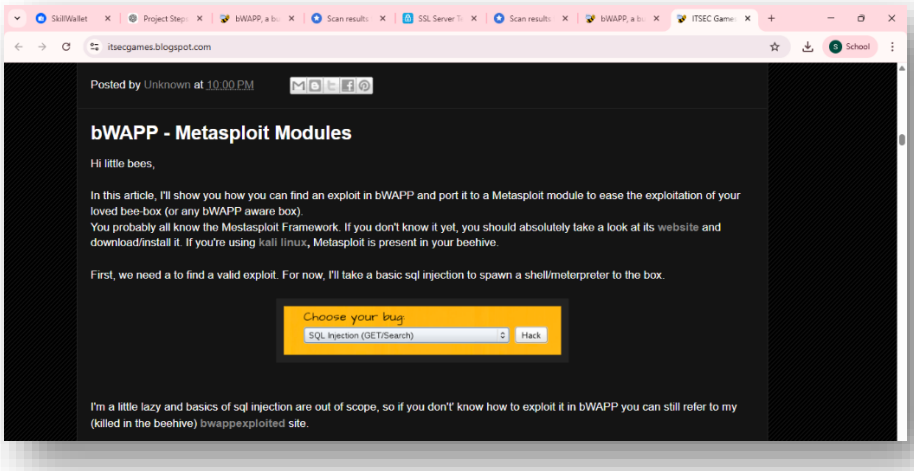

Figure 5: SQL Injection vulnerability demonstrated through manipulated GET search input.

Recommendation:

- Use prepared statements or parameterised queries
- Implement input validation and sanitisation
- Apply the principle of least privilege to database accounts
- Use Web Application Firewalls (WAF) to detect injection attempts

4. Vulnerability Name: Command Injection (Remote Command Execution)

CWE: CWE-78 – Improper Neutralisation of Special Elements in OS Commands

OWASP Category: A03:2021 – Injection

Description:

- Command injection occurs when user inputs are passed directly into system commands without proper validation. Attackers can exploit this vulnerability by injecting malicious commands into application inputs.
- During testing, a command execution vulnerability was identified where user input is passed to the server's command line interface. By manipulating the input parameters, attackers may execute system commands on the server.

Business Impact:

This vulnerability may allow attackers to execute arbitrary commands, access sensitive system files, install malware, or gain full control of the server.

Evidence:

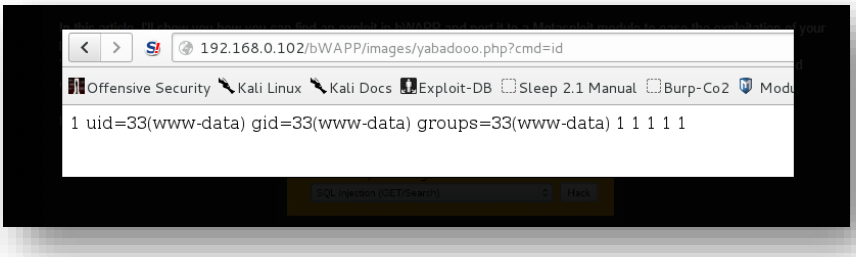

Figure 6: Command injection vulnerability allowing execution of system commands on the server.

Recommendation:

- Avoid executing system commands using user input
- Implement strict input validation
- Use secure coding practices
- Restrict system command execution permissions

5. Vulnerability Name: Open Ports and Service Exposure

CWE: CWE-284 – Improper Access Control

OWASP Category: A05:2021 – Security Misconfiguration

Description:

- Network port scanning revealed that several services are exposed on the target server. Open ports indicate that certain services are running and accessible from external networks.
- While some ports are necessary for server functionality, exposing unnecessary services can increase the attack surface of the system.

Business Impact:

Attackers may attempt to exploit vulnerabilities in exposed services or perform brute-force attacks on network services. This may lead to unauthorised system access.

Evidence:

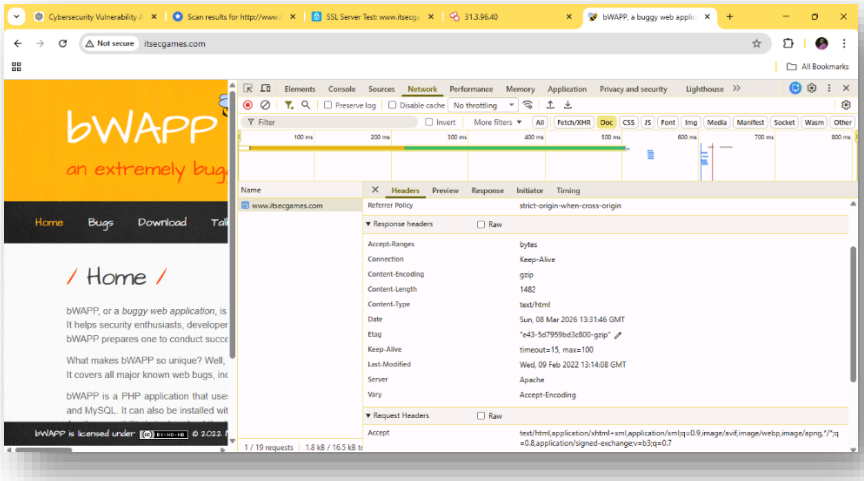

Figure 7: Network scan results showing open ports and exposed services on the target server.

Recommendation:

- Close unused network ports
- Restrict access using firewall rules
- Regularly monitor open ports and services
- Implement network intrusion detection systems

6. Vulnerability Name: Server Information Disclosure

CWE: CWE-200 – Exposure of Sensitive Information

OWASP Category: A05:2021 – Security Misconfiguration

Description:

- Server information disclosure occurs when a web server reveals details about its software configuration through HTTP response headers. These headers may reveal the server type, version, and other technical details.
- During testing, the HTTP response headers revealed information about the web server software being used by the application.

Business Impact:

Attackers may use this information to identify known vulnerabilities associated with the server software and perform targeted attacks.

Evidence:

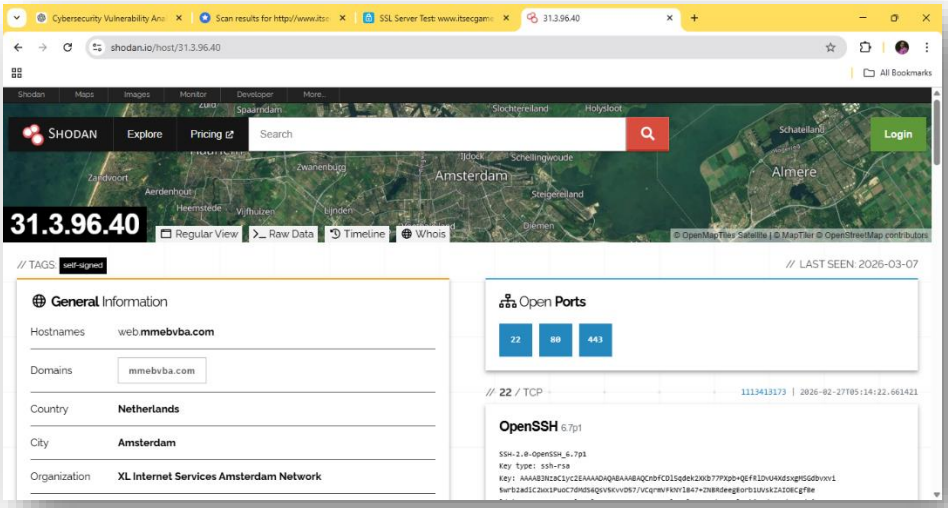

Figure 8: HTTP response headers revealing web server information and configuration details.

Recommendation:

- Disable server version disclosure in configuration files
- Remove unnecessary HTTP headers
- Keep server software updated with security patches

7. Vulnerability Name: Technology Information Disclosure

CWE: CWE-200 – Exposure of Sensitive Information

OWASP Category: A05:2021 – Security Misconfiguration

Description:

- Technology fingerprinting tools were used to analyse the technologies used by the web application. The scan results revealed information about the server platform, web server software, and additional services used by the application.
- This information can help attackers identify the technology stack used by the application and search for vulnerabilities associated with those technologies.

Business Impact:

Attackers may use technology fingerprinting information to identify known vulnerabilities and launch targeted attacks against the system.

Evidence:

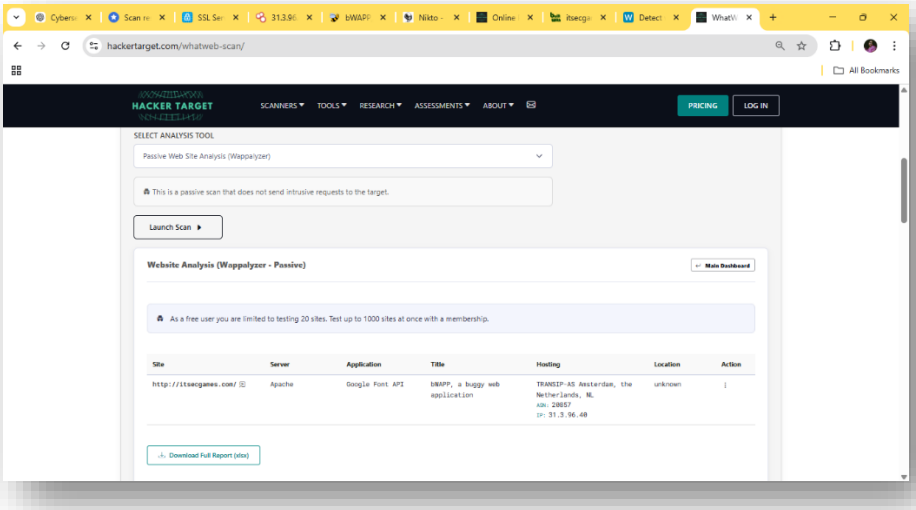

Figure 9: Technology fingerprinting results identifying the server platform and application technologies.

Recommendation:

- Limit technology information exposure
- Disable unnecessary server response headers
- Implement proper security configurations

8. Vulnerability Name: Cross-Site Scripting (XSS)

CWE: CWE-79 – Improper Neutralisation of Input During Web Page Generation
OWASP Category: A03:2021 – Injection

Description:

- Cross-Site Scripting (XSS) is a common web application vulnerability that occurs when an application allows untrusted user input to be included in web pages without proper validation or output encoding.
- In this attack, malicious scripts are injected into trusted web pages and executed in the victim's browser. Attackers typically exploit input fields such as search boxes, login forms, or comment sections.
- During the security testing process, the web application was analysed to determine whether user input was properly sanitised. Improper handling of user input may allow attackers to insert malicious JavaScript code that runs when other users access the affected page.
- If exploited, the attacker's script will execute in the user's browser and may interact with the web application as if it were the legitimate user.

Business Impact:

Successful exploitation of XSS vulnerabilities may allow attackers to steal session cookies, capture sensitive information, redirect users to malicious websites, or perform unauthorised actions on behalf of the victim.

Evidence:

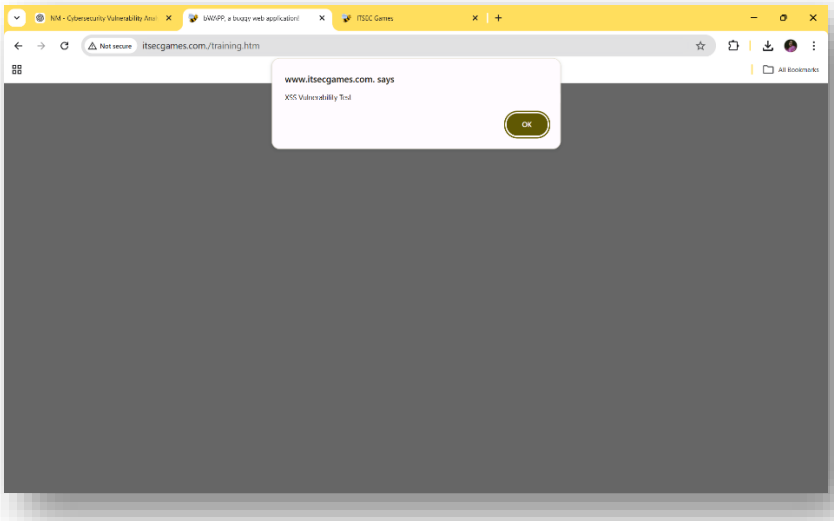

Figure 10: Demonstration of Cross-Site Scripting (XSS) using browser developer console

Recommendation:

- Validate and sanitise all user inputs.
- Implement proper output encoding for dynamic content.
- Use Content Security Policy (CSP) to restrict execution of malicious scripts.
- Apply secure development practices to prevent injection vulnerabilities.

9. Vulnerability Name: Insecure Cookie Configuration

CWE: CWE-614 – Sensitive Cookie in HTTPS Session Without Secure Attribute
OWASP Category: A05:2021 – Security Misconfiguration

Description:

- Cookies are commonly used by web applications to store session identifiers and user-related information in the browser.
- If cookies are not configured with appropriate security attributes such as **Secure, HttpOnly, and SameSite**, attackers may be able to intercept or manipulate them.
- During the vulnerability assessment, cookie storage was inspected using **Chrome Developer Tools (Application → Cookies)** to verify the presence of cookies and their security attributes.
- As shown in the evidence screenshot, **no cookies were found for the tested page (http://www.itsecgames.com)**, indicating that the application does not currently store cookies for this page.

Business Impact:

If cookies were implemented without proper security attributes, attackers could potentially hijack user sessions and impersonate legitimate users. This may lead to unauthorised access to sensitive data or application functionalities.

Evidence:

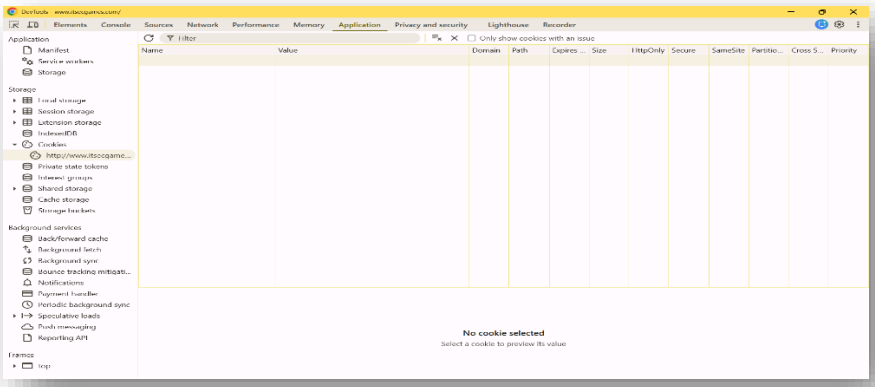

Figure 11: Cookie Inspection Using Chrome Developer Tools

The screenshot shows the inspection of cookies for the **Google website** using **Chrome Developer Tools in the Application → Cookies section**. It displays several cookies along with their attributes, such as **Name, Value, Domain, Expiry, HttpOnly, Secure, and SameSite**, which are used to analyse how cookies are stored and secured in the browser.

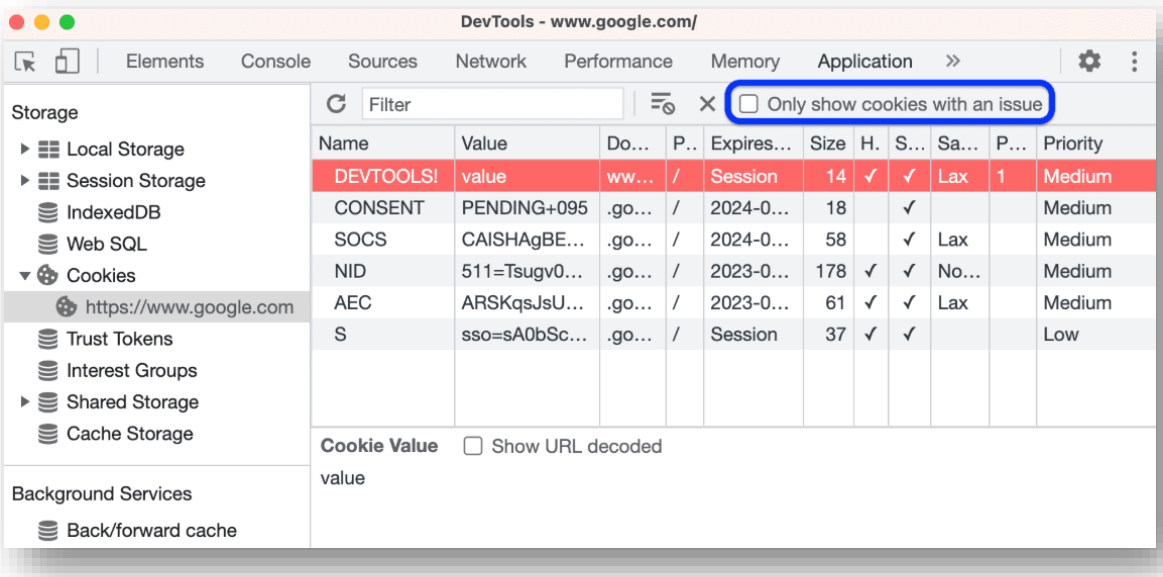

Figure 13: Cookie inspection of the Google website using Chrome Developer Tools.

Recommendation:

- Ensure that any cookies implemented in the future include appropriate security attributes such as **Secure, HttpOnly, and SameSite**.
- Use **HTTPS** to protect cookie transmission over the network.
- Avoid storing sensitive information directly in cookies.
- Implement proper **session management practices** to reduce the risk of session hijacking.

10. Vulnerability Name: Clickjacking Vulnerability

CWE: CWE-1021 – Improper Restriction of Rendered UI Layers
OWASP Category: A05:2021 – Security Misconfiguration

Description:

- Clickjacking is a web-based attack where an attacker tricks users into clicking on hidden elements within a web page by embedding the target website inside an invisible frame.

- If the web application does not implement protection mechanisms such as the X-Frame-Options header or Content Security Policy frame restrictions, attackers may be able to load the application within malicious websites.
- During the vulnerability assessment, the HTTP response headers were analysed to verify whether frame protection mechanisms were implemented.
- The absence of frame protection headers indicates that the application may be vulnerable to clickjacking attacks.

Business Impact:

Attackers may trick users into performing unintended actions such as submitting forms, transferring funds, or changing account settings without the user's knowledge.

Evidence:

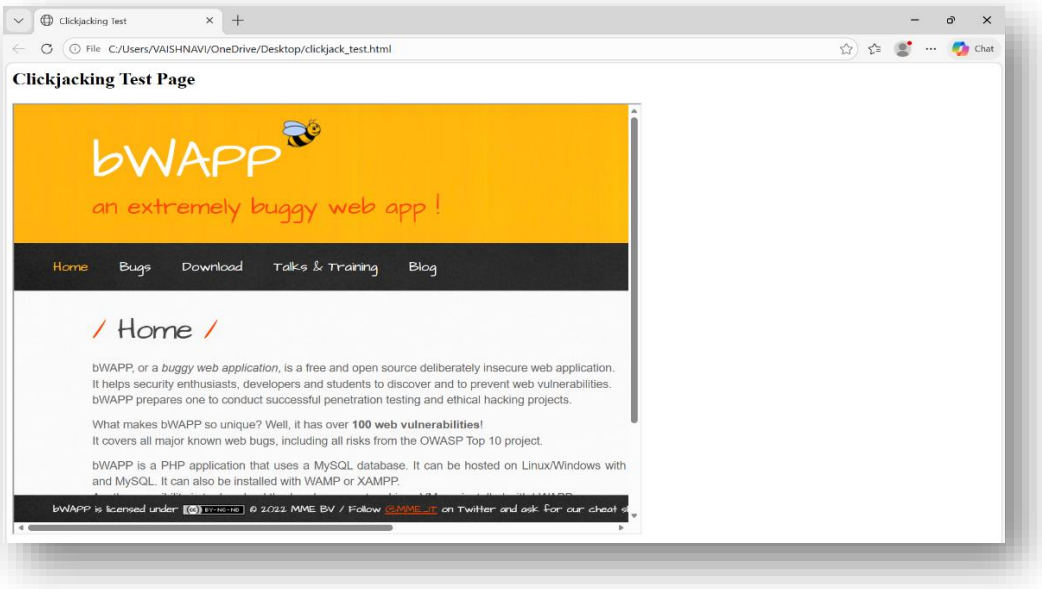

Figure 13: Clickjacking Vulnerability Test Using Clickjacking Test Tool

Recommendation:

- Implement **X-Frame-Options** header with values such as DENY or SAMEORIGIN.
- Use **Content Security Policy (CSP) frame-ancestors directive** to restrict framing.
- Conduct regular security configuration reviews of HTTP headers.

STAGE: 2

Vulnerability Scanning Using Nessus

Overview

Nessus is a widely used vulnerability assessment tool developed by Tenable. It is designed to identify security weaknesses in computer systems, networks, and web applications by performing automated vulnerability scans. Nessus helps security professionals detect misconfigurations, outdated software, open ports, weak encryption protocols, and other potential security risks that may expose systems to cyber attacks.

The primary purpose of Nessus is to perform vulnerability scanning and generate detailed reports that help organisations understand their security posture. Nessus works using a large database of vulnerability checks called plugins. These plugins contain information about known security vulnerabilities, software bugs, configuration errors, and security misconfigurations. During the scanning process, Nessus compares the target system against this database to detect possible vulnerabilities.

Nessus classifies vulnerabilities into different severity levels such as Critical, High, Medium, Low, and Informational. Critical vulnerabilities represent the most serious security threats that could allow attackers to gain unauthorised access or compromise the system. Medium and low vulnerabilities indicate potential security weaknesses that should be addressed to prevent exploitation. Informational vulnerabilities provide system information that may assist attackers in planning further attacks.

In this project, Nessus Essentials was used to perform vulnerability scanning on the target system. The scanning process involved identifying open ports, detecting running services, analysing operating system information, and checking for known vulnerabilities using plugin-based detection techniques. Nessus also performs service detection and system fingerprinting to identify the technologies running on the target server.

After completing the scan, Nessus generates a detailed vulnerability report showing the list of vulnerabilities detected in the target system along with severity levels, plugin identification numbers, vulnerability descriptions, and recommended solutions. This information helps security analysts understand the weaknesses present in the system and take appropriate remediation measures to improve security.

The vulnerability assessment performed using Nessus successfully detected several vulnerabilities in the target system, including a critical vulnerability related to an outdated Ubuntu Linux operating system, as well as multiple informational vulnerabilities related to SSH configuration, HTTP services, and service detection. By analysing these vulnerabilities, organisations can strengthen their security infrastructure and protect their systems from potential cyber threats.

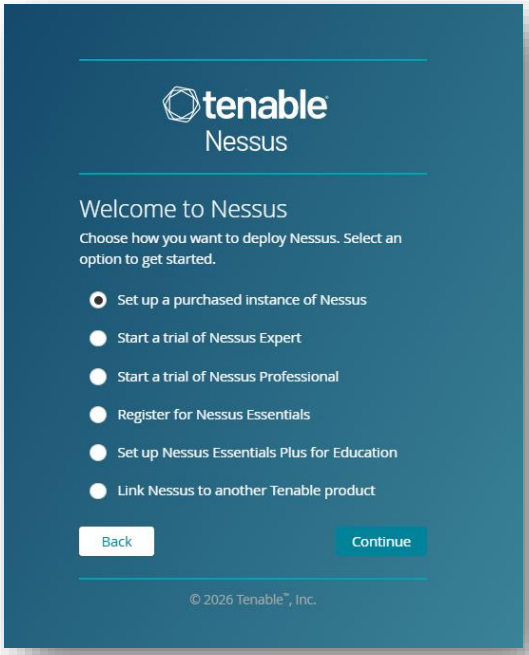

Figure 1: Initial Setup Interface of Tenable Nessus Showing Deployment Options

Target Website: scanme.nmap.org

Target IP Address: 45.33.32.156

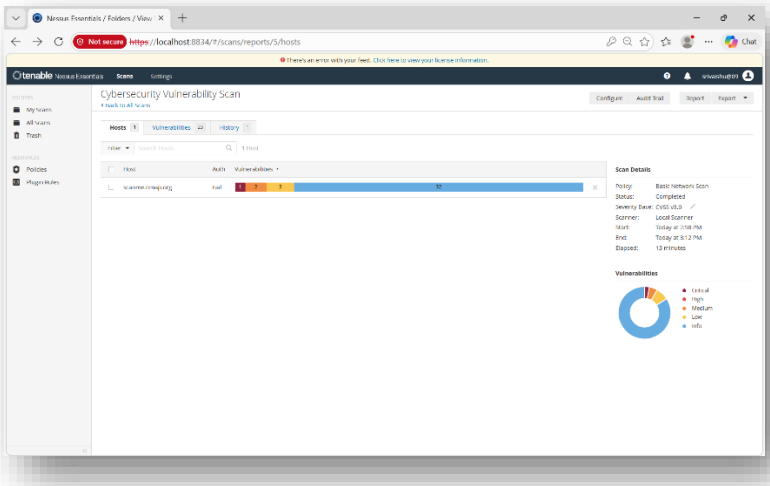

Figure 2: Nessus Scan Result Showing Detected Vulnerabilities and Severity Classification

List of Vulnerabilities

S.No	Vulnerability Name	Severity	Plugin
1	Canonical Ubuntu Linux SEoL (14.04.x)	Critical	201408
2	SSH Weak Algorithms Supported	Medium	90317
3	SSH Server CBC Mode Ciphers Enabled	Low	70658
4	SSH Weak Key Exchange Algorithms Enabled	Low	153953
5	SSH Weak MAC Algorithms Enabled	Low	71049
6	SSH Algorithms and Languages Supported	Info	70657
7	SSH SHA-1 HMAC Algorithms Enabled	Info	153588
8	HTTP Methods Allowed (Per Directory)	Info	43111
9	Apache Banner Linux Distribution Disclosure	Info	18261
10	Nessus SYN Scanner	Info	11219

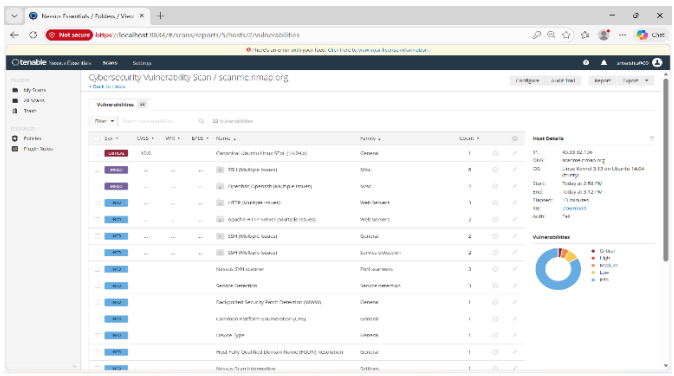

Figure 3: Nessus Vulnerability List Detected During Scan

1. Vulnerability Name: Canonical Ubuntu Linux SEoL (14.04.x)

Severity: Critical

Plugin: 201408

Port: 80/tcp (www)

Description:

The Nessus vulnerability scan identified that the target system is running **Ubuntu Linux version 14.04**, which has reached its **Security End of Life (SEoL)**. According to the scan results, Ubuntu 14.04 stopped receiving security updates on **April 24, 2019**. This means that the vendor no longer maintains the operating system and does not receive any security patches or updates.

Operating systems that reach end-of-life status become highly vulnerable to cyberattacks because vendors do not fix newly discovered security vulnerabilities. Attackers can exploit these vulnerabilities to gain unauthorised access, perform remote attacks, or compromise system resources.

Because the system runs an outdated operating system version, it may contain known vulnerabilities that attackers could exploit. Nessus classifies this vulnerability as **Critical** because outdated operating systems pose a significant security risk to the overall system infrastructure.

The scan output also confirms that the detected operating system is **Ubuntu Linux 14.04**, which has been unsupported for more than six years.

Solution:

The recommended solution is to **upgrade the operating system to a currently supported version of Ubuntu Linux**. Supported versions such as **Ubuntu 20.04 LTS or Ubuntu 22.04 LTS** receive regular security updates, patches, and vendor support.

System administrators should ensure that outdated operating systems are replaced with supported versions and implement a regular patch management strategy to maintain system security.

Business Impact:

Using an unsupported operating system exposes the organisation to serious security risks. Attackers may exploit known vulnerabilities in outdated systems to gain unauthorised access to the network or server.

Successful exploitation could lead to data breaches, system compromise, service disruption, and financial losses for the organisation. Therefore, maintaining updated operating systems is essential to protect sensitive information and ensure system security.

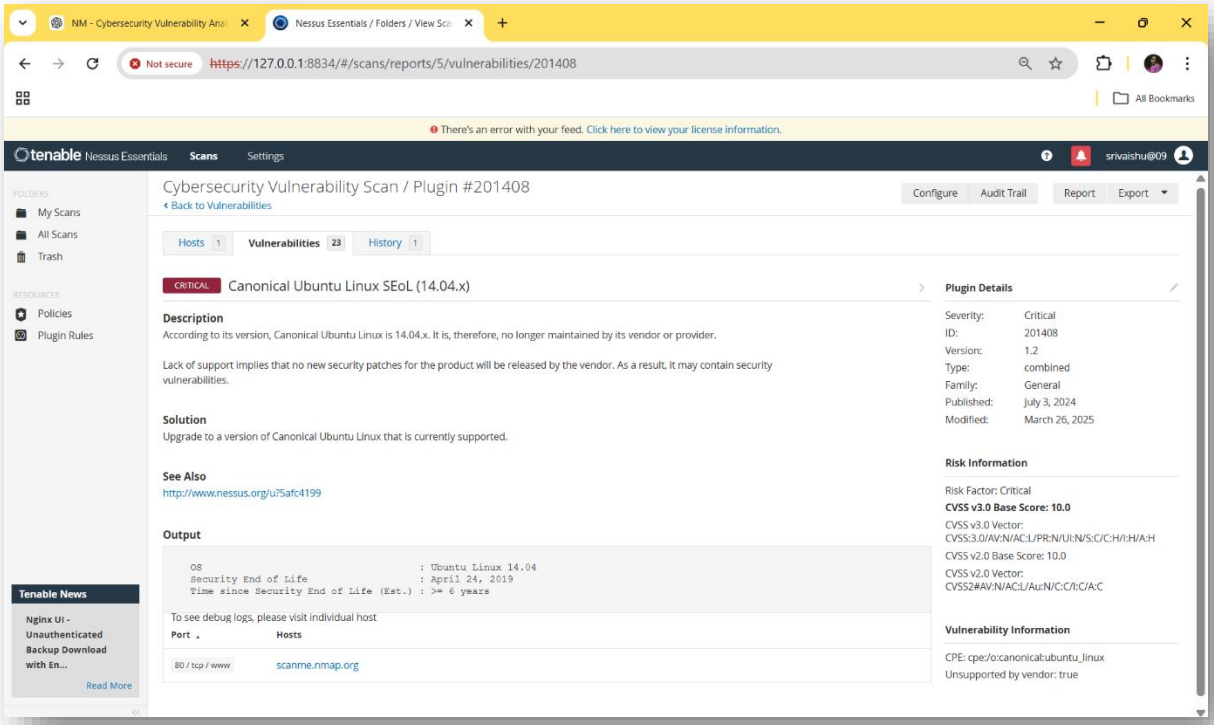

Figure 4: Critical Vulnerability Detected – Canonical Ubuntu Linux SEoL (14.04.x)

2. Vulnerability Name: SSH Weak Algorithms Supported

Severity: Medium

Plugin: 90317

Port: 22/tcp (ssh)

Description:

The Nessus scan detected that the SSH service running on the target system supports **weak encryption algorithms**. These weak algorithms include outdated cryptographic methods such as the **Arcfour stream cypher**, which is considered insecure due to weaknesses in its key generation and encryption mechanism.

According to **RFC 4253**, the use of weak encryption algorithms like Arcfour is not recommended because they can compromise the security of encrypted communications. If attackers are able to exploit these weak algorithms, they may be able to intercept or decrypt sensitive data transmitted between the SSH client and the server.

SSH (Secure Shell) is commonly used for secure remote login and administration of systems. If weak encryption algorithms are enabled in the SSH configuration, attackers may attempt cryptographic attacks to weaken the security of the communication channel.

This vulnerability is classified as **Medium severity** because it does not immediately allow system compromise, but weakens the security of the encrypted communication.

Solution:

To mitigate this vulnerability, system administrators should **disable weak encryption algorithms in the SSH configuration**. Only strong and modern encryption algorithms such as **AES-256, AES-192, or ChaCha20** should be enabled.

Administrators should also update the SSH server to the latest version and configure secure cryptographic settings in the SSH configuration file (sshd_config).

Regular security audits and configuration reviews should be performed to ensure that weak cryptographic algorithms are not used.

Business Impact:

If weak encryption algorithms are used, attackers may attempt to exploit these cryptographic weaknesses to intercept or decrypt secure communications. This may expose sensitive information such as login credentials, administrative commands, or confidential system data.

Compromised SSH communication could lead to unauthorised access, data leakage, or system manipulation by attackers. Therefore, disabling weak encryption algorithms is essential to maintain strong communication security.

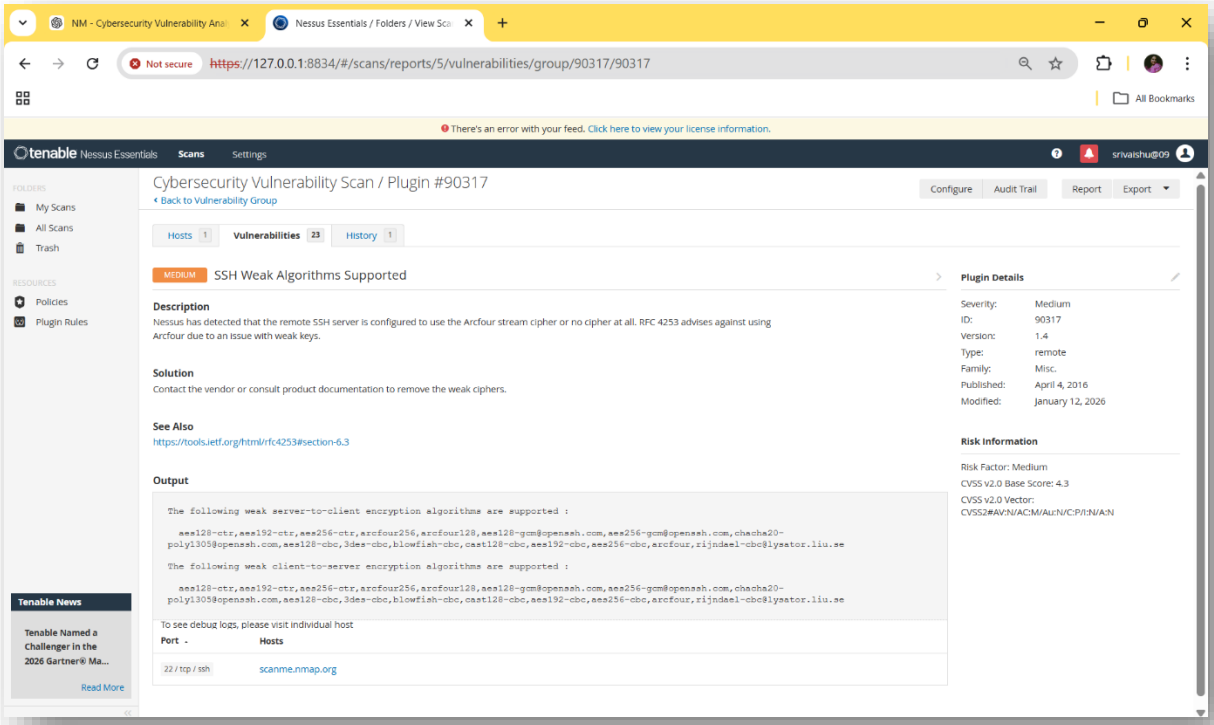

Figure 5: Nessus Detected SSH Weak Encryption Algorithms Vulnerability

3. Vulnerability Name: SSH Server CBC Mode Ciphers Enabled

Severity: Low

Plugin: 70658

Port: 22/tcp (SSH)

Description

The vulnerability scan performed using Nessus identified that the SSH server running on the target system supports **Cipher Block Chaining (CBC) mode encryption**. CBC mode encryption is considered less secure compared to modern encryption techniques and may allow attackers to recover portions of the plaintext message from the ciphertext.

CBC mode encryption has known weaknesses, particularly related to **chosen-plaintext attacks and message manipulation vulnerabilities**. Attackers may exploit these weaknesses to perform cryptographic attacks against the SSH communication channel.

Although CBC encryption still provides a certain level of security, it is considered outdated compared to stronger encryption modes such as **CTR (Counter Mode) and GCM (Galois/Counter Mode)**. Modern security best practices recommend disabling CBC mode ciphers in SSH configurations to prevent potential exploitation.

In this case, the Nessus plugin detected that the SSH service running on **port 22** supports CBC mode ciphers, which may expose the system to cryptographic vulnerabilities during secure communications.

Solution:

To remediate this vulnerability, system administrators should **disable CBC mode encryption algorithms in the SSH configuration**. Instead, the system should use more secure encryption algorithms such as **AES-CTR, AES-GCM, or ChaCha20-Poly1305**.

Administrators can modify the SSH configuration file (sshd_config) to remove CBC ciphers and only allow strong encryption methods. After making configuration changes, the SSH service should be restarted to apply the new security settings.

Regular system updates and security configuration reviews should also be performed to ensure that outdated encryption algorithms are not enabled.

Business Impact:

If CBC mode encryption remains enabled, attackers may attempt to exploit its cryptographic weaknesses to analyse encrypted SSH traffic. This could potentially lead to **information disclosure, session manipulation, or partial recovery of sensitive data** transmitted through the SSH connection.

Although the severity level is categorised as **Low**, leaving outdated encryption mechanisms enabled increases the attack surface of the system and may weaken the overall security posture of the network.

Therefore, disabling CBC mode encryption is recommended to strengthen the security of remote access communications.

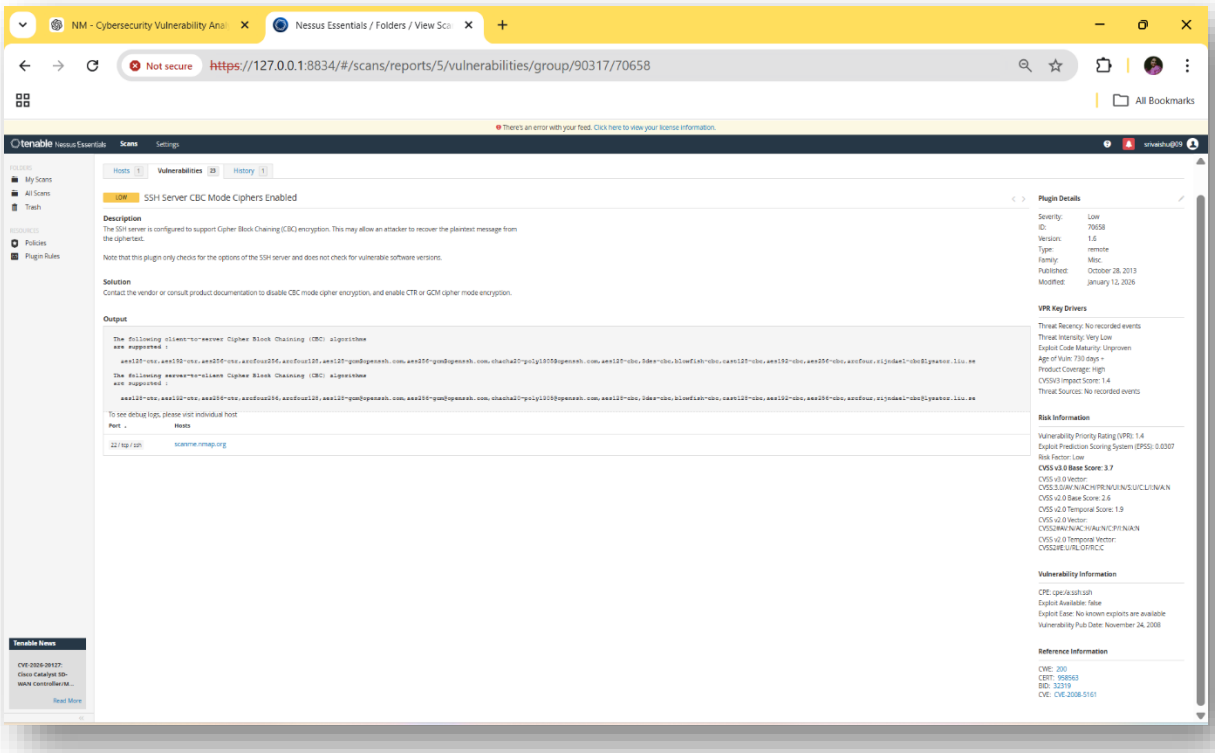

Figure 6: Nessus Detection of SSH Server CBC Mode Cipher Vulnerability

4. Vulnerability Name: SSH Weak Key Exchange Algorithms Enabled

Severity: Low

Plugin: 153953

Port: 22/tcp (SSH)

Description:

The vulnerability scan performed using Nessus detected that the SSH server on the target system supports **weak key exchange algorithms**. Key Exchange (KEX) algorithms are responsible for securely establishing encryption keys between the client and the server during the SSH connection process.

The presence of weak key exchange algorithms indicates that the SSH server allows outdated cryptographic mechanisms such as **diffie-hellman-group-exchange-sha1, diffie-hellman-group1-sha1, gss-group1-sha1, and rsa1024-sha1**. These algorithms are considered insecure according to modern cryptographic standards.

This vulnerability is based on recommendations provided in **IETF RFC 9142**, which states that weak key exchange algorithms should not be enabled because they can be exploited through cryptographic attacks. Attackers may attempt to weaken the encryption process and potentially intercept or manipulate secure communication.

Although this vulnerability does not immediately compromise the system, it weakens the overall security of SSH communications and increases the risk of cryptographic attacks.

Solution:

To mitigate this vulnerability, system administrators should **disable weak SSH key exchange algorithms** and configure the server to support only strong and modern cryptographic algorithms.

The SSH configuration file (sshd_config) should be updated to remove weak key exchange algorithms and allow secure algorithms such as:

- diffie-hellman-group14-sha256
- ecdh-sha2-nistp256
- ecdh-sha2-nistp384
- ecdh-sha2-nistp521
- curve25519-sha256

After modifying the SSH configuration file, the SSH service should be restarted to apply the changes. Regular security audits should also be performed to ensure that outdated cryptographic algorithms are not enabled.

Business Impact:

If weak key exchange algorithms remain enabled, attackers may attempt to exploit cryptographic weaknesses to intercept SSH communications. This may lead to **unauthorized access, data interception, or manipulation of secure sessions**.

Even though the vulnerability severity is classified as **Low**, leaving weak encryption algorithms enabled can reduce the effectiveness of secure communication channels and expose the system to potential security risks.

Maintaining strong cryptographic standards is essential to ensure secure remote access and protect sensitive system communications.

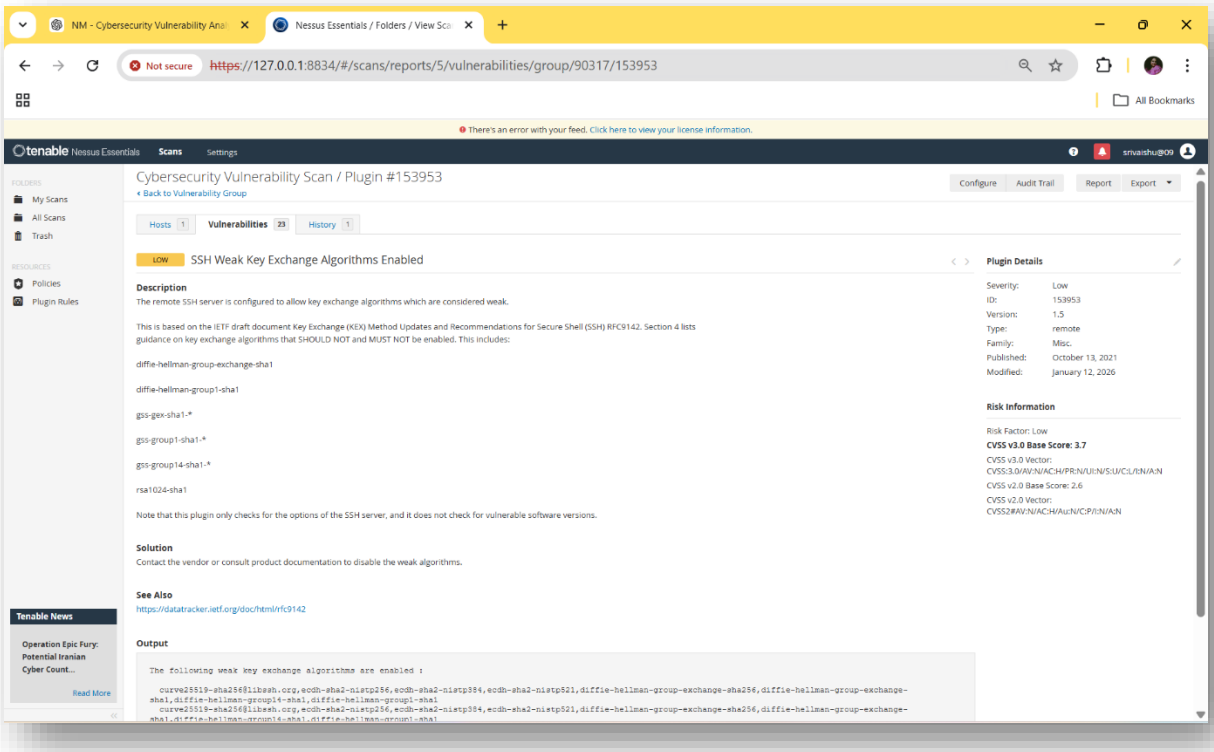

Figure 7: Nessus Detection of SSH Weak Key Exchange Algorithms Vulnerability

5. Vulnerability Name: SSH Weak MAC Algorithms Enabled

Severity: Low

Plugin: 71049

Port: 22/tcp (SSH)

Description:

The vulnerability assessment performed using the Nessus security scanner detected that the SSH server running on the target system supports **weak Message Authentication Code (MAC) algorithms**. MAC algorithms are used in Secure Shell (SSH) communication to ensure the integrity and authenticity of transmitted data between the client and the server.

The scan results indicate that the SSH server allows weak MAC algorithms such as **MD5-based or 96-bit MAC algorithms**, which are considered cryptographically insecure. These algorithms are vulnerable to collision attacks and cryptographic weaknesses that may allow attackers to manipulate or forge data packets.

When weak MAC algorithms are enabled, attackers may attempt to exploit these weaknesses to compromise the integrity of encrypted communication. This could potentially allow malicious actors to modify or tamper with data transmitted during an SSH session.

Although this vulnerability does not directly allow system compromise, it weakens the overall security of SSH communications and increases the risk of integrity attacks.

Solution:

To mitigate this vulnerability, system administrators should **disable weak MAC algorithms in the SSH configuration** and allow only strong and secure MAC algorithms.

The SSH server configuration file (sshd_config) should be updated to remove weak MAC algorithms, such as:

- hmac-md5
- hmac-md5-96
- hmac-sha1-96

Instead, administrators should enable stronger MAC algorithms, such as:

- hmac-sha2-256
- hmac-sha2-512
- umac-128-etm@openssh.com

After updating the SSH configuration file, the SSH service should be restarted to apply the changes. Regular security configuration reviews should also be conducted to ensure that outdated cryptographic algorithms are not enabled.

Business Impact:

If weak MAC algorithms remain enabled, attackers may attempt to exploit these cryptographic weaknesses to tamper with SSH communication. This could potentially lead to **data manipulation, integrity compromise, or unauthorized modification of transmitted information**.

Although the vulnerability severity is classified as **Low**, leaving weak MAC algorithms enabled reduces the overall security of the SSH communication channel and increases the potential attack surface.

Ensuring strong cryptographic configurations is essential to maintain secure remote access and protect sensitive system communications.

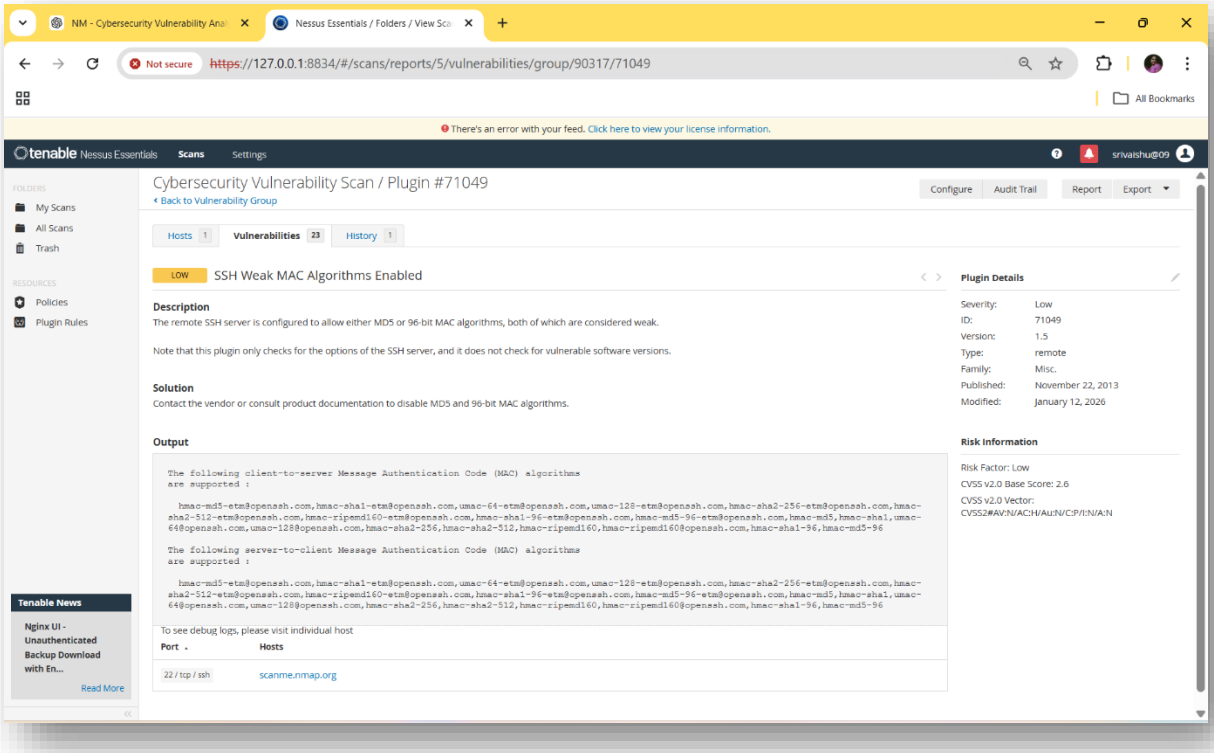

Figure 8: Nessus Detection of SSH Weak Message Authentication Code (MAC) Algorithms

6. Vulnerability Name: SSH Algorithms and Languages Supported

Severity: Informational

Plugin: 70657

Port: 22/tcp (SSH)

Description:

The Nessus vulnerability scanner detected the **encryption algorithms and language options supported by the SSH service** running on the target system. This plugin does not identify a direct vulnerability but provides informational details about the cryptographic algorithms that the SSH server supports for secure communication.

During the scanning process, Nessus negotiated several encryption algorithms with the remote SSH server. The results show that the server supports encryption algorithms such as **AES256-CTR** for both client-to-server and server-to-client communication. These algorithms are responsible for encrypting data transmitted during SSH sessions.

The scan also detected the compression algorithms supported by the SSH service. These options determine whether data compression is enabled during encrypted communication between the client and the server.

This information helps security administrators understand the cryptographic configuration of the SSH server and evaluate whether strong encryption standards are being used.

Although this finding is categorised as **Informational**, it provides valuable insights into the security configuration of the SSH service.

Solution:

Since this plugin is informational, no direct remediation is required. However, system administrators should periodically review the supported SSH encryption algorithms to ensure that only **secure and modern cryptographic algorithms** are enabled.

It is recommended to disable outdated or weak encryption algorithms and allow only strong algorithms such as:

- AES-256-CTR
- AES-256-GCM
- ChaCha20-Poly1305

Regular security configuration reviews and vulnerability assessments should be performed to maintain secure SSH communication.

Business Impact:

Although this finding does not represent an immediate security threat, understanding the supported encryption algorithms is important for maintaining secure communication channels.

If weak algorithms are enabled in the future, attackers may exploit them to compromise encrypted communications. Monitoring cryptographic configurations ensures that the system follows modern security standards and reduces potential attack vectors.

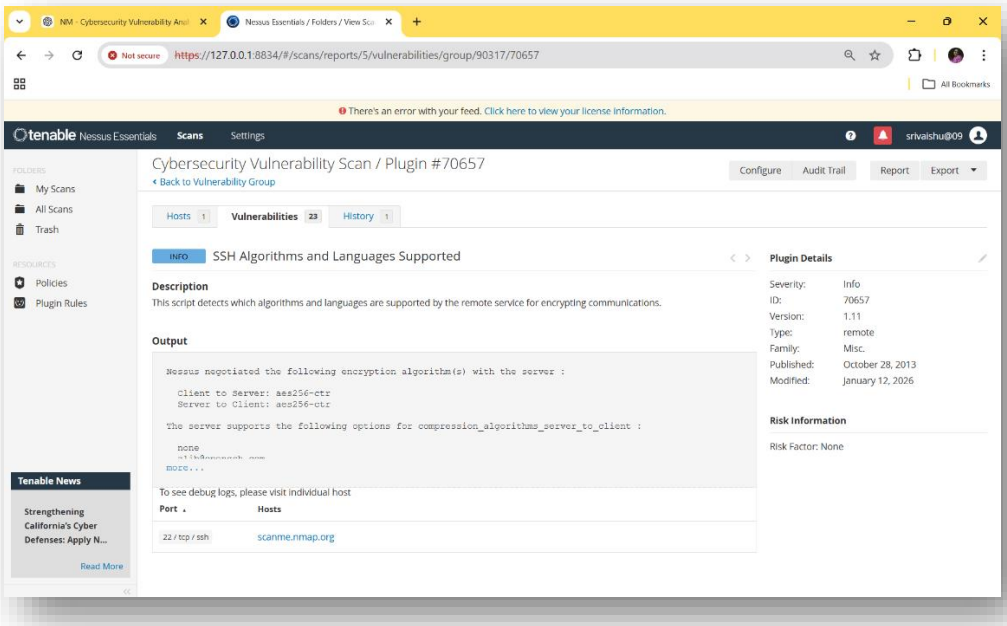

Figure 9: Nessus Detection of Supported SSH Encryption Algorithms

7. Vulnerability Name: SSH SHA-1 HMAC Algorithms Enabled

Severity: Informational

Plugin: 153588

Port: 22/tcp (SSH)

Description:

The Nessus vulnerability scanner detected that the SSH server running on the target system supports **SHA-1-based HMAC (Hash-based Message Authentication Code) algorithms**. HMAC algorithms are used in SSH communication to verify the integrity and authenticity of transmitted data between the client and the server.

SHA-1 was previously widely used as a secure hashing algorithm. However, due to advances in cryptographic research, SHA-1 is now considered **deprecated for many security purposes**, especially digital signatures, because of its susceptibility to collision attacks. Although SHA-1 is still considered relatively secure for HMAC operations, modern security standards recommend using stronger hashing algorithms.

The scan results indicate that the SSH server allows SHA-1-based HMAC algorithms such as **hmac-sha1** and related variants. These algorithms are gradually being replaced by stronger cryptographic algorithms like **SHA-256 and SHA-512**.

This plugin primarily detects the configuration of the SSH server and identifies which HMAC algorithms are supported during secure communication sessions.

Solution:

Since SHA-1-based HMAC algorithms are considered outdated, system administrators should **update the SSH configuration to prioritise stronger cryptographic algorithms**.

The SSH configuration file (sshd_config) should be modified to disable SHA-1-based HMAC algorithms and enable stronger alternatives, such as:

- hmac-sha2-256
- hmac-sha2-512
- umac-128-etm@openssh.com

After modifying the configuration file, the SSH service should be restarted to apply the changes. Regular security assessments should also be performed to ensure that outdated cryptographic algorithms are not enabled in the system.

Business Impact:

Although this finding is categorised as **Informational**, the use of outdated cryptographic algorithms may weaken the security of SSH communication over time. If attackers exploit weaknesses in older hashing algorithms, it could potentially compromise the integrity of secure communications.

Using stronger hashing algorithms ensures better protection of transmitted data and helps maintain compliance with modern cybersecurity standards and best practices.

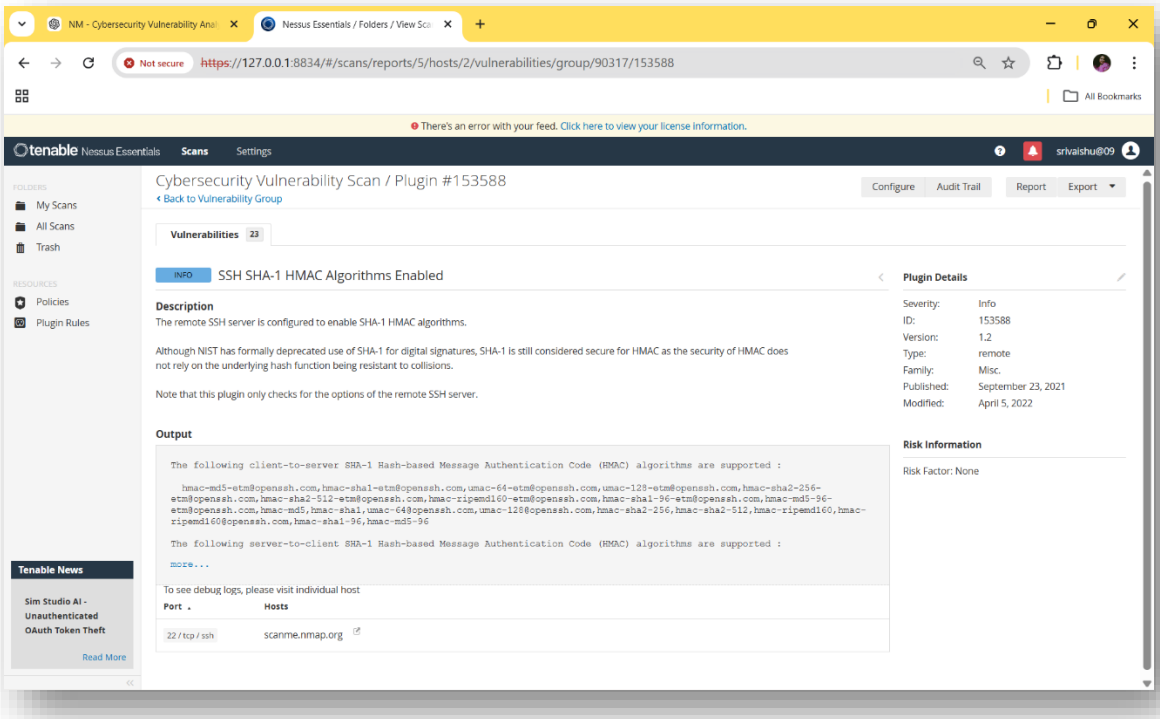

Figure 9: Nessus Detection of SSH SHA-1 HMAC Algorithms Enabled

8. Vulnerability Name: HTTP Methods Allowed (Per Directory)

Severity: Informational

Plugin: 43111

Port: 80/tcp (HTTP)

Description:

The Nessus vulnerability scanner detected the **HTTP methods that are allowed by the web server for specific directories**. HTTP methods define the types of requests that a client can send to a web server to interact with web resources.

During the vulnerability scan, the server responded to an **OPTIONS request**, which allows Nessus to identify the HTTP methods that are supported by the server. The scan results indicate that the web server allows the following HTTP methods:

- GET
- HEAD
- OPTIONS
- POST

Some HTTP methods, such as **PUT, DELETE, CONNECT, TRACE, and HEAD,** can potentially introduce security risks if they are not properly restricted. For example, attackers may attempt to exploit certain HTTP methods to upload malicious files, manipulate server resources, or bypass authentication controls.

In this case, the scan result shows that the server allows standard HTTP request methods. The information collected helps administrators understand the configuration of the web server and determine whether unnecessary or insecure methods are enabled.

This finding is categorised as **Informational**, as it does not directly indicate a vulnerability but provides useful information about the server configuration.

Solution:

Although this finding is informational, it is recommended that system administrators **restrict unnecessary HTTP methods** to reduce potential security risks.

Web server configurations should allow only the methods required for application functionality. Typically, secure configurations allow only:

- GET
- POST
- HEAD

Administrators should review the web server configuration (such as **Apache or Nginx configuration files**) and disable unnecessary HTTP methods like **PUT, DELETE, TRACE, and CONNECT** unless they are explicitly required.

Regular security audits should be performed to ensure that only necessary HTTP methods are enabled.

Business Impact:

Allowing unnecessary HTTP methods may increase the attack surface of the web application. Attackers may attempt to exploit these methods to perform unauthorised actions such as **uploading malicious files, bypassing security controls, or manipulating server resources**.

Even though this finding is categorised as **Informational**, restricting unused HTTP methods improves the overall security posture of the web server and helps prevent potential exploitation.

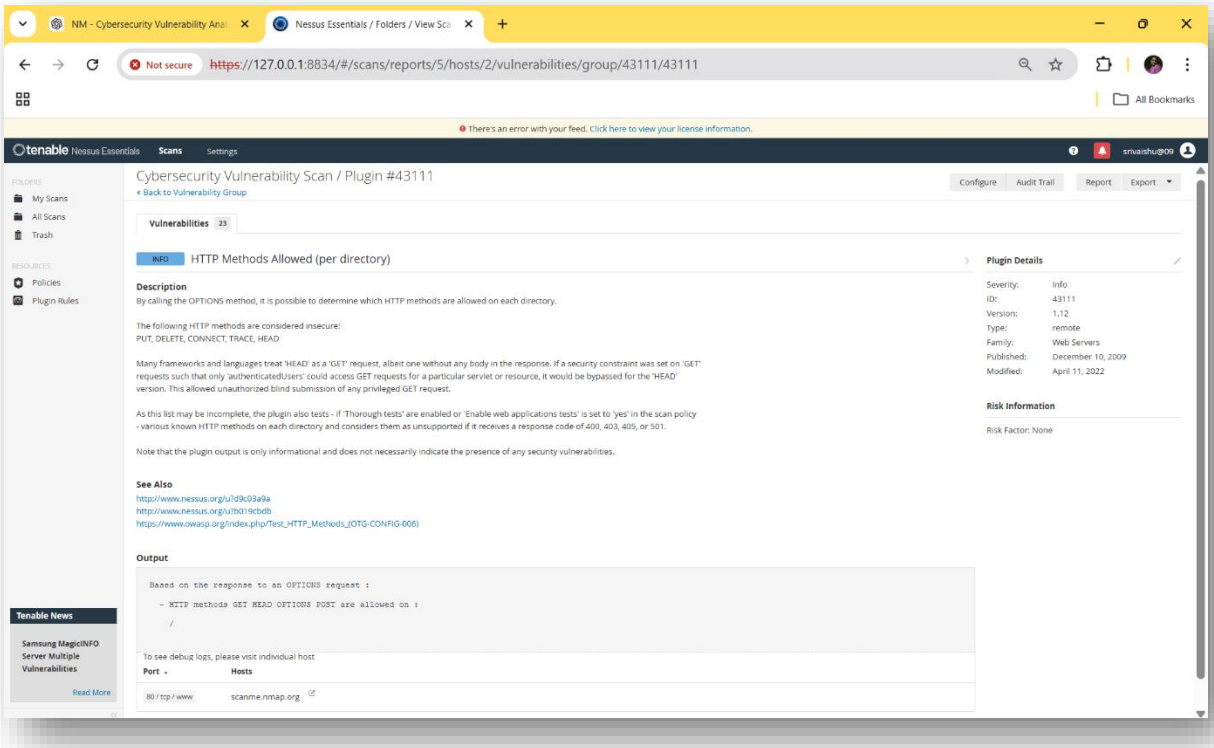

Figure 10: Nessus Detection of Allowed HTTP Methods on the Web Server

9. Vulnerability Name: Apache Banner Linux Distribution Disclosure

Severity: Informational

Plugin: 18261

Service: Apache Web Server

Description:

The vulnerability scan performed using the Nessus security scanner detected that the **Apache web server is revealing information about the underlying Linux distribution through its server banner**.

A server banner is a message returned by a web server in response to client requests. This banner often includes details about the **web server software, version number, and operating system** running on the server. In this case, the scan identified that the server is running **Ubuntu Linux 14.04 (Trusty)**.

When such information is publicly exposed, attackers may use it for **information gathering during the reconnaissance phase of an attack**. Knowing the operating system and server version can help attackers identify known vulnerabilities associated with that specific software version.

Although this finding is categorised as **Informational**, it can assist attackers in performing targeted attacks by providing insight into the system environment.

Solution:

To prevent server information disclosure, administrators should **hide or minimise server banner information** in the Apache configuration.

This can be achieved by modifying the Apache configuration file (httpd.conf or apache2.conf) and setting the following directive:

- ServerTokens Prod
- ServerSignature Off

The **ServerTokens Prod** directive ensures that Apache only returns minimal information about the server without revealing detailed version or operating system information.

After modifying the configuration file, the Apache service should be **restarted** for the changes to take effect.

Regular security configuration reviews should also be conducted to ensure that sensitive system information is not exposed.

Business Impact:

Although this issue does not directly compromise the system, **information disclosure increases the risk of targeted attacks**. Attackers can use the disclosed server information to identify vulnerabilities associated with the specific operating system or web server version.

By hiding server banners and limiting system information exposure, organisations can reduce the amount of useful information available to attackers and improve the overall security posture of the system.

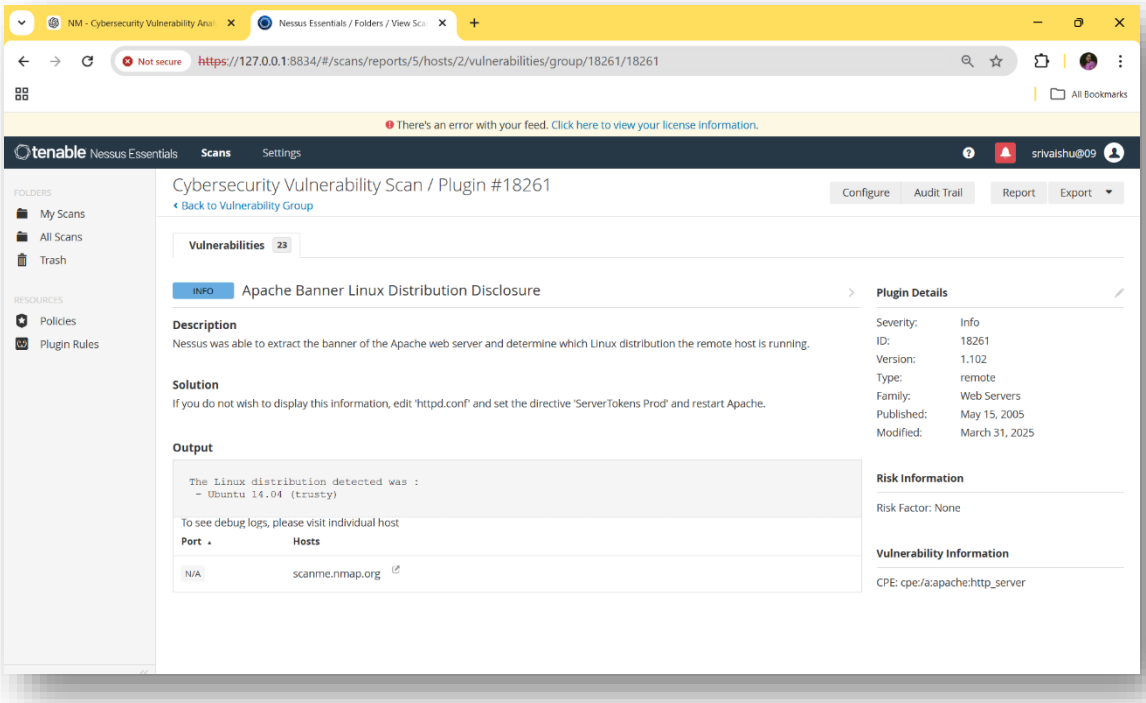

Figure 11: Nessus Detection of Apache Server Banner Linux Distribution Disclosure

10. Vulnerability Name: Nessus SYN Scanner – Open Port Detection

Severity: Informational

Plugin: 11219

Type: Port Scanning

Description:

The Nessus vulnerability scanner detected open ports on the target host using the **SYN scanning technique**. A SYN scan, also known as a **half-open port scan**, is a method used to determine whether specific ports on a remote system are open, closed, or filtered.

During the scanning process, Nessus sends a **SYN packet** to the target port. If the server responds with a **SYN-ACK packet**, the port is considered open and accepting connections. If an **RST packet** is returned, the port is considered closed. This technique allows the scanner to identify open services on the system without completing the full TCP handshake.

The scan results identified that the following ports are open on the target host:

- Port 22/tcp – SSH (Secure Shell service)
- Port 80/tcp – HTTP (Web server service)
- Port 31337/tcp – Unknown or custom service

Open ports indicate that services are actively running on the system and are accessible over the network. While open ports are necessary for system functionality, they may also expose services that could potentially be exploited if they contain vulnerabilities.

Solution:

To reduce potential security risks, system administrators should ensure that **only necessary ports and services are exposed to the network**.

The following security measures are recommended:

- Close unused or unnecessary ports on the system.
- Configure **firewalls or network access control rules** to restrict access to essential services only.
- Regularly monitor open ports using vulnerability scanning tools.
- Apply security updates and patches to services running on open ports.

Implementing proper network filtering and firewall rules can help protect systems from unauthorised access and potential attacks.

Business Impact:

Open ports increase the **attack surface of a system** because they expose services that may be targeted by attackers. If vulnerable services are running on open ports, attackers may attempt to exploit them to gain unauthorised access or compromise the system.

Even though this finding is categorised as **Informational**, monitoring and controlling open ports is an important aspect of network security management. Ensuring that only necessary services are accessible helps strengthen the security posture of the system.

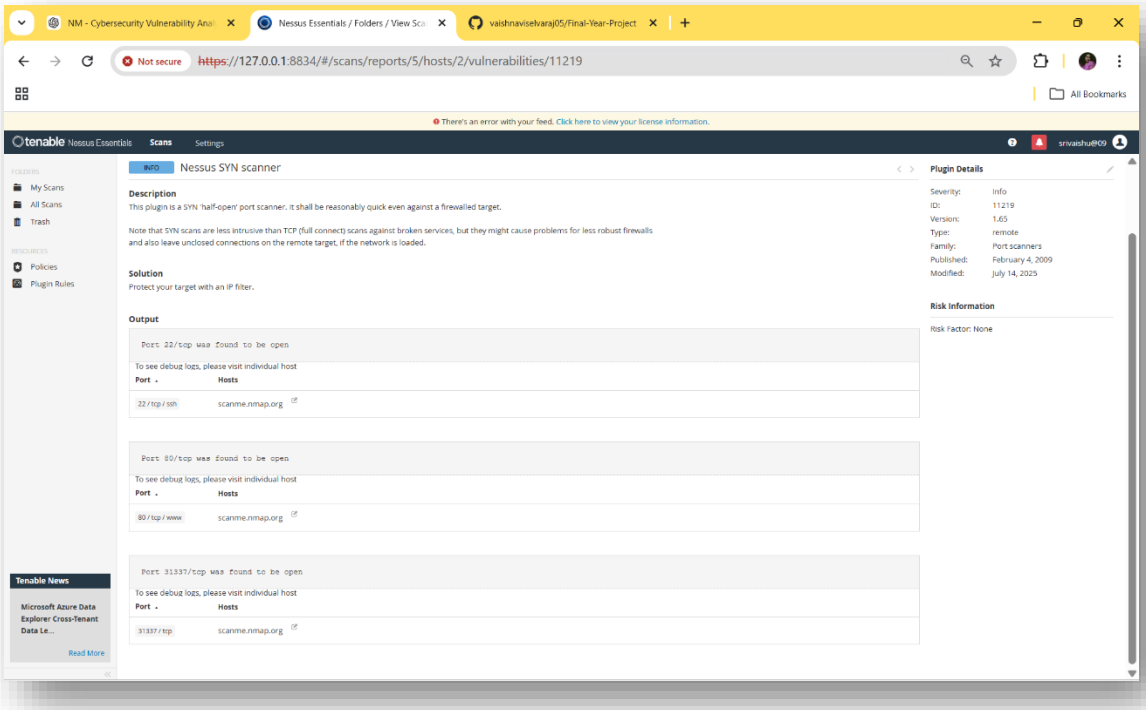

Figure 12: Nessus SYN Scan Showing Open Ports on the Target System

STAGE: 3

Implementation and Understanding of Security Operations Centre (SOC) and SIEM for Organisational Cybersecurity Monitoring

SOC (Security Operations Centre)

A **Security Operations Centre (SOC)** is a centralised facility where cybersecurity professionals monitor, detect, analyse, and respond to security incidents across an organisation's network, systems, and applications. The SOC team is responsible for maintaining the security posture of an organisation by continuously monitoring logs, network traffic, and system activities.

The main purpose of a SOC is to provide **24/7 security monitoring and incident management**. SOC analysts use various security tools such as SIEM platforms, intrusion detection systems (IDS), intrusion prevention systems (IPS), firewalls, and threat intelligence platforms to identify suspicious activities.

SOC teams are typically structured into different roles such as **Level 1 analysts (monitoring and alert triage), Level 2 analysts (incident investigation), and Level 3 analysts (advanced threat hunting and response)**. These teams work together to detect potential cyber threats and prevent attacks such as malware infections, data breaches, denial-of-service attacks, and unauthorised access.

The SOC also plays a critical role in maintaining **organisational security policies, incident response procedures, and compliance requirements**. By continuously monitoring the environment, SOC helps organisations detect security threats early and respond quickly before they cause significant damage.

SOC Cycle

The **SOC cycle** represents the continuous workflow followed by security analysts to detect and respond to cyber threats effectively. The cycle typically consists of several stages.

The first stage is **data collection**, where logs and security events are gathered from multiple sources such as servers, firewalls, network devices, and applications. These logs provide important information about activities happening within the system.

The next stage is **monitoring and detection**, where SOC analysts monitor the incoming data using SIEM tools. Suspicious patterns or abnormal activities, such as multiple failed login attempts or unusual network traffic, are identified.

After detection, the SOC team performs an **investigation and analysis**. Analysts examine the logs, correlate events, and determine whether the activity is malicious or a false positive.

Once a threat is confirmed, the **incident response** phase begins. The SOC team takes appropriate actions to contain the attack, remove malicious components, and prevent further damage.

The final stage is **recovery and improvement**, where affected systems are restored, and lessons learned from the incident are used to improve future security defences.

SIEM (Security Information and Event Management)

Security Information and Event Management (SIEM) is a cybersecurity solution that collects, aggregates, and analyses security logs from different sources within an organisation. SIEM systems play a critical role in helping SOC teams monitor network activity and detect potential threats.

A SIEM platform gathers log data from devices such as **servers, routers, switches, firewalls, antivirus systems, and applications**. These logs contain valuable information about user activities, system events, and network traffic.

The SIEM system processes this data and performs **correlation analysis**, which means it looks for relationships between different events. For example, if multiple login failures occur followed by a successful login from a suspicious location, the SIEM system may flag this as a potential security threat.

SIEM tools also provide **real-time alerts, dashboards, and reporting features**, allowing SOC analysts to quickly identify and investigate security incidents. Additionally, SIEM systems support compliance reporting for security standards such as ISO 27001 and PCI-DSS.

Overall, SIEM is an essential component of modern cybersecurity operations because it provides centralised visibility and automated threat detection.

SIEM Cycle

The **SIEM cycle** describes how security information is collected, processed, and analysed to detect threats.

The first step in the cycle is **data collection**, where logs are gathered from various sources, including operating systems, network devices, security tools, and cloud services.

The collected data then undergoes **log normalisation**, where different log formats are standardised so they can be analysed effectively.

The next stage is **event correlation**, where the SIEM system analyses patterns across multiple logs to identify suspicious behaviour. This helps detect advanced attacks that may not be visible from a single log source.

After correlation, the system performs **analysis and alert generation**. If unusual activity is detected, alerts are generated and sent to SOC analysts for investigation.

Finally, SOC analysts perform **incident response and reporting**, where they investigate the alerts, confirm whether an attack has occurred, and take necessary actions to mitigate the threat.

MISP (Malware Information Sharing Platform)

MISP (Malware Information Sharing Platform) is an open-source threat intelligence platform used to share cybersecurity threat data between organisations. It allows security teams to exchange information about malicious activities such as malware infections, phishing campaigns, and suspicious IP addresses.

MISP stores and organises **Indicators of Compromise (IOCs)** such as malicious domains, IP addresses, file hashes, and URLs. These indicators help security teams identify potential threats in their networks.

One of the key advantages of MISP is **collaboration**. Organisations can share threat intelligence with trusted communities, enabling faster detection of cyber attacks. By using shared threat data, security teams can proactively block malicious activities before they cause harm.

MISP is widely used by **government agencies, security researchers, and organisations** to improve global cybersecurity awareness and defence capabilities.

College Network Information

A typical college network consists of multiple interconnected systems, including **administrative offices, academic departments, computer laboratories, and student networks**. These systems are connected through network devices such as routers, switches, and firewalls.

The college network also includes servers that host **web applications, databases, student portals, and internal management systems**. Students and staff access these services through wired and wireless networks.

Because many users access the network daily, the college infrastructure generates a large volume of system logs and network traffic. Monitoring this activity is important to detect unauthorised access attempts, malware infections, or suspicious network behaviour.

Implementing proper cybersecurity monitoring in the college network helps protect sensitive information such as **student records, academic data, and administrative documents**.

How SOC Can Be Deployed in the College

Deploying a SOC in a college environment involves establishing centralised security monitoring and incident response capabilities. The first step is to collect logs from different systems such as servers, firewalls, routers, and applications.

These logs can be integrated into a **SIEM platform** that analyses security events and generates alerts when suspicious activities are detected.

Security analysts can monitor the SIEM dashboard to detect threats such as unauthorised login attempts, malware infections, or network scanning activities.

The SOC team can also implement **security policies, vulnerability management, and threat intelligence integration** to strengthen the college's cybersecurity posture.

By deploying a SOC, the college can improve its ability to detect cyber attacks early, respond quickly to incidents, and protect its digital infrastructure.

Threat Intelligence

Threat intelligence refers to the collection and analysis of information about cyber threats that may affect an organisation. This information includes details about attackers, malware types, vulnerabilities, and attack techniques.

Threat intelligence helps security teams understand the tactics used by cybercriminals and anticipate potential attacks. By analysing threat intelligence data, organisations can strengthen their defences and implement preventive security measures.

Threat intelligence sources include **security research reports, threat intelligence platforms, vulnerability databases, and information shared by security communities**.

Integrating threat intelligence into SOC operations helps analysts identify emerging threats and respond proactively.

Incident Response

Incident response is the structured process used to manage and respond to cybersecurity incidents. It ensures that organisations can quickly identify and mitigate security threats while minimising damage.

The incident response process usually includes several stages, such as **preparation, detection, analysis, containment, eradication, recovery, and post-incident review**.

During an incident, the SOC team investigates alerts generated by security monitoring systems. If a threat is confirmed, the team isolates affected systems, removes malicious components, and restores normal operations.

After resolving the incident, security teams analyse the event to identify weaknesses in the system and improve security controls.

QRadar & Understanding About the Tool

IBM QRadar is an advanced SIEM platform used by organisations to monitor and analyse security events in real time. It collects log data from various sources and applies correlation rules to detect suspicious activities.

QRadar provides powerful features such as **real-time threat detection, network traffic analysis, automated alert generation, and interactive dashboards**. These dashboards help SOC analysts visualise security events and quickly identify potential threats.

The tool also supports **behavioural analysis and anomaly detection**, allowing organisations to identify unusual user activities or network patterns.

QRadar is widely used in large organisations because it provides centralised security monitoring and helps SOC teams respond to cyber incidents effectively.

Conclusion

Stage 1 – What You Understand from Web Application Testing

In Stage 1, we gained practical knowledge about web application security testing and vulnerability identification. Web applications are widely used for online services, making them a common target for cyber attacks. During this stage, we learned how to analyse web applications and identify security vulnerabilities that may expose sensitive data or system functionality.

We explored different types of vulnerabilities such as SQL Injection, Command Injection, Cross-Site Scripting (XSS), Missing Security Headers, Clickjacking, and Insecure Cookie Configuration. By performing vulnerability testing on web applications, we understood how attackers exploit these weaknesses to gain unauthorised access or manipulate application behaviour.

This stage helped us understand the importance of secure coding practices, proper input validation, and strong security configurations in web applications. It also demonstrated how security testing tools and manual testing techniques can be used to identify potential vulnerabilities before attackers exploit them.

Overall, Stage 1 provided a strong foundation in web application security assessment and vulnerability analysis, which is an important skill for cybersecurity professionals.

Stage 2 – What You Understand from the Nessus Report

In Stage 2, we worked with Nessus, a widely used vulnerability scanning tool that helps identify security weaknesses in systems and networks. Nessus scans the target system and generates detailed reports about detected vulnerabilities, including their severity levels, descriptions, and recommended mitigation measures.

Through this stage, we learned how automated vulnerability scanners analyse systems for misconfigurations, outdated software, weak encryption settings, exposed services, and other security issues. The Nessus report provides valuable insights into potential risks and helps security teams prioritise remediation efforts.

We also understood how vulnerability assessment plays an important role in proactive cybersecurity management. By identifying and addressing vulnerabilities early, organisations can significantly reduce the risk of cyber attacks and data breaches.

This stage helped us understand how security professionals use vulnerability scanning tools as part of risk management and security auditing processes.

Stage 3 – What You Understand from SOC / SIEM / QRadar Dashboard

In Stage 3, we learned about the Security Operations Centre (SOC) and Security Information and Event Management (SIEM) systems used to monitor and manage cybersecurity threats in real time. We explored how security tools such as IBM QRadar collect and analyse logs from different systems and network devices.

The SOC environment provides centralised monitoring where security analysts continuously track security events, investigate alerts, and respond to potential incidents. SIEM platforms play a critical role by correlating security logs and detecting suspicious patterns that may indicate cyber attacks.

Through the QRadar dashboard, we understood how security events are visualised and how analysts can quickly identify abnormal activities within the network. This stage demonstrated the importance of real-time monitoring, threat detection, and incident response in maintaining a strong security posture.

Overall, Stage 3 helped us understand how modern organisations use SOC and SIEM solutions to detect cyber threats early, respond effectively to incidents, and maintain continuous security monitoring.

Future Scope

Stage 1 – Future Scope of Web Application Testing

Web application security testing will continue to evolve as web technologies become more advanced and complex. Future developments may include the use of Artificial Intelligence (AI) and Machine Learning (ML) to automatically detect vulnerabilities in web applications.

Security testing tools will become more sophisticated, enabling automated detection of vulnerabilities during the software development lifecycle. The adoption of DevSecOps practices will ensure that security testing is integrated into every stage of application development.

Additionally, future research may focus on improving techniques for detecting advanced vulnerabilities such as business logic flaws, API vulnerabilities, and cloud-based security issues.

Stage 2 – Future Scope of Testing Process

The vulnerability testing process will continue to evolve with the integration of continuous security testing and automated scanning tools. Organisations will increasingly adopt automated security testing within their development pipelines to identify vulnerabilities earlier.

Advanced vulnerability scanners will incorporate AI-driven analysis, behavioural monitoring, and threat intelligence integration to improve detection accuracy. This will help organisations identify complex vulnerabilities that traditional scanners may miss.

Future testing processes may also involve automated remediation systems, where vulnerabilities detected by scanning tools are automatically fixed or patched using predefined security policies.

Stage 3 – Future Scope of SOC / SIEM

The future of SOC and SIEM technologies will involve greater integration with Artificial Intelligence, Machine Learning, and advanced analytics. These technologies will help security teams detect sophisticated cyber attacks more accurately and reduce false positives.

Modern SOC environments are also evolving toward Security Orchestration, Automation, and Response (SOAR) platforms, which automate many incident response activities. This will enable faster detection and response to cyber threats.

Cloud-based SIEM solutions will also become more common as organisations migrate their infrastructure to cloud environments. These solutions will provide scalable security monitoring for both on-premise and cloud systems.

In the future, SOC operations will rely heavily on automation, advanced threat intelligence sharing, and predictive analytics to proactively identify and prevent cyber attacks before they impact organisational systems.

END